

Clase 148 — Análisis de comportamiento

Parte: **6 — Análisis de malware** · Fuente: *Learning Malware Analysis* (Monnappa K. A.) y MITRE ATT&CK  Duración estimada: **120 min** · Nivel: **Avanzado**

Objetivo

Reconstruir el **comportamiento completo** de una muestra y mapearlo a MITRE ATT&CK: persistencia, escalada, evasión de defensas, inyección de código, movimiento y exfiltración. Se integran análisis dinámico, forense de memoria y desensamblado para pasar de "hace cosas" a un relato estructurado de TTPs que alimenta detección y respuesta.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Mapear** las acciones observadas a tácticas y técnicas de ATT&CK con su ID.
2. **Identificar** mecanismos de persistencia y priorizar los más sigilosos.
3. **Reconocer** técnicas de inyección de código (DLL/APC/hollowing).
4. **Correlacionar** memoria, procesos y red para un relato coherente.
5. **Producir** una matriz de comportamiento reutilizable en detección.

Temas

#	Tema	Por qué importa
1	Ciclo de vida del malware en el host	Da estructura al análisis
2	Persistencia (Run keys, servicios, tareas, WMI)	Continuidad tras reinicio
3	Inyección de código (varias técnicas)	Evasión y ejecución en otro proceso
4	Evasión de defensas	Deshabilita AV/logs, vive en memoria
5	Forense de memoria con Volatility	Ve lo que el disco oculta
6	Mapeo a MITRE ATT&CK	Lenguaje común de detección
7	Matriz de comportamiento	Convierte análisis en detecciones

Definiciones y características

- **Persistencia:** mecanismo para sobrevivir reinicios. Característica clave: **múltiples ASEPs** posibles.
- **Process injection:** ejecutar código en otro proceso. Característica clave: **evasión y camuflaje**.
- **Process hollowing:** vaciar un proceso legítimo y sustituir su imagen. Característica clave: **suplantación**.
- **DLL injection / APC injection:** cargar código vía DLL o colas APC. Característica clave: **ejecución remota** en un proceso.

- **TTP:** táctica, técnica y procedimiento. Característica clave: **abstracción estable** frente a IOCs volátiles.

Herramientas y preparación

- **ProcMon, Process Hacker, Autoruns** (persistencia y actividad).
- **Volatility 3** para análisis de memoria (`pslist`, `malfind`, `dlllist`, `handles`).
- **Ghidra/x64dbg** para confirmar la técnica en el código.
- Matriz **MITRE ATT&CK** abierta como referencia.

 **Nota ética y de seguridad:** la detonación y captura de memoria se realizan en la VM aislada con snapshot. Estudiar técnicas de inyección o persistencia es para **defender y detectar**; nunca despliegues estas técnicas fuera de sistemas propios y autorizados.

Laboratorio guiado

1. Restaura `base-clean` e instrumenta la VM (ProcMon, Process Hacker, Wireshark, INetSim).
2. Detona la muestra y captura un volcado de memoria (snapshot de RAM del hypervisor o `winpmem`).
3. Con **Volatility 3**: `windows.pslist` y `windows.pstree` para el árbol de procesos; localiza procesos hijos anómalos.
4. Ejecuta `windows.malfind` para hallar regiones de memoria RWX inyectadas y procesos "huecos" (hollowing). Anota PIDs.
5. Con `windows.dlllist` y `windows.handles`, identifica DLLs inyectadas y el mutex de infección.
6. Localiza la persistencia: `Autoruns` en vivo + `RegSetValue` en ProcMon. Clasifícala (Run key, servicio, tarea, WMI).
7. Para cada acción, asigna la técnica ATT&CK correspondiente (p. ej. `T1055` Process Injection, `T1547.001` Registry Run Keys, `T1053.005` Scheduled Task).
8. Construye una **matriz de comportamiento**: táctica · técnica (ID) · evidencia · posible detección.

Ejercicios

1. Diferencia DLL injection, hollowing y APC injection con un ejemplo de cada uno.
2. Mapea 6 acciones observadas a técnicas ATT&CK con su ID.
3. Explica cómo `malfind` detecta código inyectado.
4. Propón una regla de detección (Sysmon/EDR) para la persistencia hallada.
5. Correlaciona un proceso de `pslist` con su conexión de red.
6. Justifica por qué los TTPs son más duraderos que los IOCs.

Reto verificable

Entrega una **matriz de comportamiento** de una muestra con al menos 8 técnicas ATT&CK (con ID y evidencia), la inyección confirmada por `malfind` y la persistencia identificada, más una detección propuesta por técnica. **Criterio de aceptación:** cada técnica tiene evidencia (ProcMon/Volatility/Wireshark) y un ID `Txxxx` correcto, y al menos una detección propuesta es implementable con Sysmon o un EDR.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>malfind</code> sin resultados	Inyección ya cerrada o en otro proceso; captura memoria antes
Persistencia "no existe"	Solo miraste Run keys; revisa servicios, tareas y WMI
Confundir técnica con IOC	El IP es IOC; "C2 over HTTPS" es la técnica ATT&CK
Árbol de procesos confuso	Usa <code>pstree</code> ; correlaciona PPID con el proceso padre real
IDs de ATT&CK inventados	Verifica cada técnica en attack.mitre.org

Preguntas frecuentes

 **¿Por qué mapear a ATT&CK?** Porque estandariza el reporte, permite comparar amenazas y priorizar detecciones sobre técnicas que el atacante no puede cambiar fácilmente.

 **¿Memoria o disco?** Ambos. El malware fileless o inyectado apenas toca disco; la memoria revela lo que el análisis estático no ve.

 **¿Cuántos ASEPs de persistencia existen?** Decenas. Autoruns cataloga muchos; por eso no basta con revisar las Run keys.

Referencias

- Monnappa K. A., *Learning Malware Analysis*, cap. 7–8.
- MITRE ATT&CK. <https://attack.mitre.org>
- Volatility 3. <https://github.com/volatilityfoundation/volatility3>
- Sysmon (Sysinternals). <https://learn.microsoft.com/sysinternals/downloads/sysmon>

Siguiente clase

Clase 149 - Comunicaciones de comando y control (C2) del malware